นาย นฤเศรษฐ์ อภิลักขิตพงศ์ 673380044-4 part1-3
นาย คณิศร มาประจักษ์ 673380031-3 part4-5

🧪 Lab3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

A Cisco Networking Academy–style lab that blends classical networking with application-layer engineering. You will configure inter-VLAN routing, deploy a MIME-aware socket server, capture packets, and analyze how meaning flows through the stack.

📘 Lab Overview

Scenario

A client in one VLAN transfers files to a server in another VLAN using a custom MIME-based protocol (JSON header + binary payload). You will verify connectivity, observe encapsulation, and analyze packets using Wireshark.

This lab reinforces how Layer 2 segmentation, Layer 3 routing, TCP reliability, and application metadata interact.

🎯 Objectives

After completing this lab, you will be able to:

- Configure Router-on-a-Stick inter-VLAN routing
- Configure VLANs and trunking on a switch
- Deploy and run a MIME socket server and client
- Capture traffic using Wireshark
- Identify ARP, VLAN tags, TCP handshake, and application data
- Decode JSON headers in packet payload
- Troubleshoot network and application failures

- Document protocol behavior across OSI layers

🧰 Required Resources

- 1 Router (e.g., Cisco Systems ISR/virtual router)
- 1 Switch (Layer 2)
- 2 PCs or VMs
- Ethernet cables or virtual links
- Python 3 installed
- MIME file transfer project
- Wireshark installed (from Wireshark Foundation)

🗺️ Topology

```
None
PC1 (VLAN 10) ----\
                    \ access
                     \
                     [S1]---- trunk ----[R1]
                     /
                    / access
PC2 Server (VLAN 20)
```

📋 Addressing Scheme

Device	Interface	IP	Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

🧠 Background Theory

Router-on-a-Stick

A single physical interface carries multiple VLANs via 802.1Q tagging. Subinterfaces perform routing between broadcast domains.

MIME Metadata

MIME labels content type (e.g., image/png). Your protocol wraps files with JSON describing:

- MIME type
- Size

Why This Matters

Real systems — APIs, storage services, message brokers — depend on this exact pattern: structured metadata + binary payload over reliable transport.

🔧 Part 1 — Configure Network Devices

Step 1 — Router Configuration

```
enable

configure terminal
hostname R1

interface g0/0
no shutdown

interface g0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0
```

```
end
write memory
```

```
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
interface g0/0.10
                ^
% Invalid input detected at '^' marker.

R1(config-if)#interface g0/0.10
R1(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0.10, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0.10, changed state to
up

R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#exit
R1(config)#interface g0/0.20
R1(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0.20, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0.20, changed state to
up
encapsulation dot1Q 20
R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#exit
R1(config)#
R1(config)#
R1(config)#
R1(config)#
R1(config)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#write memory
Building configuration...
[OK]
R1#
```

Copy Paste

```
R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  administratively down down
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
```

Step 2 — Switch Configuration

```
None
enable
configure terminal
hostname S1

vlan 10
name CLIENTS

vlan 20
name SERVERS

interface g0/1
switchport mode trunk

interface fa0/1
switchport mode access
switchport access vlan 10

interface fa0/2
switchport mode access
switchport access vlan 20

end
write memory
```

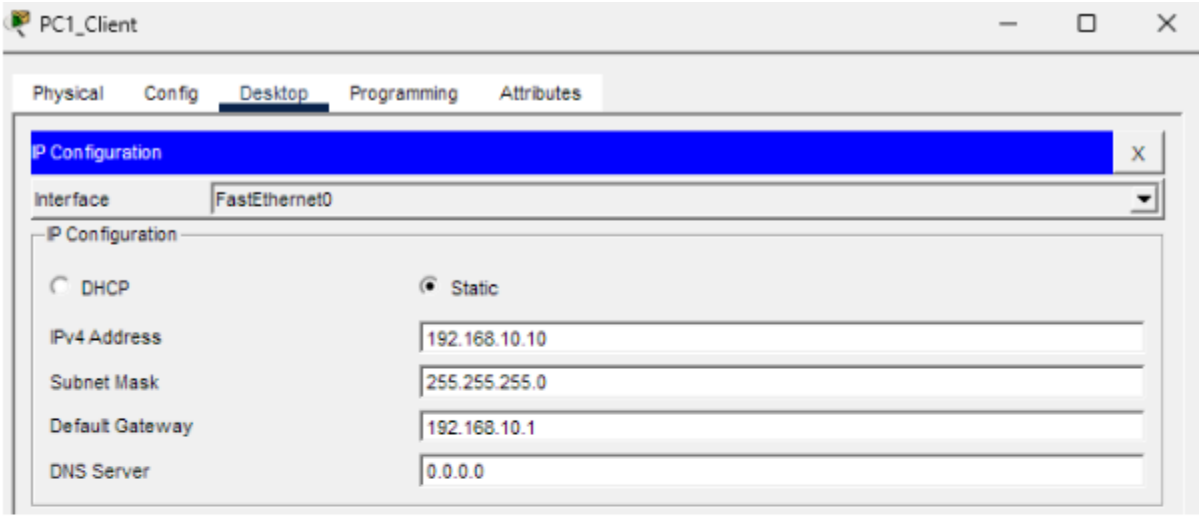
```
Switch>enable
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#exit
S1(config)#vlan 20
S1(config-vlan)#name SERVERS
S1(config-vlan)#exit
S1(config)#interface g0/1
S1(config-if)#switchport mode trunk

S1(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

S1(config-if)#exit
S1(config)#interface fa0/1
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#exit
S1(config)#interface fa0/2
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 20
S1(config-if)#exit
S1(config)#end
S1#
%SYS-5-CONFIG_I: Configured from console by console

S1#write memory
Building configuration...
[OK]
S1#
```

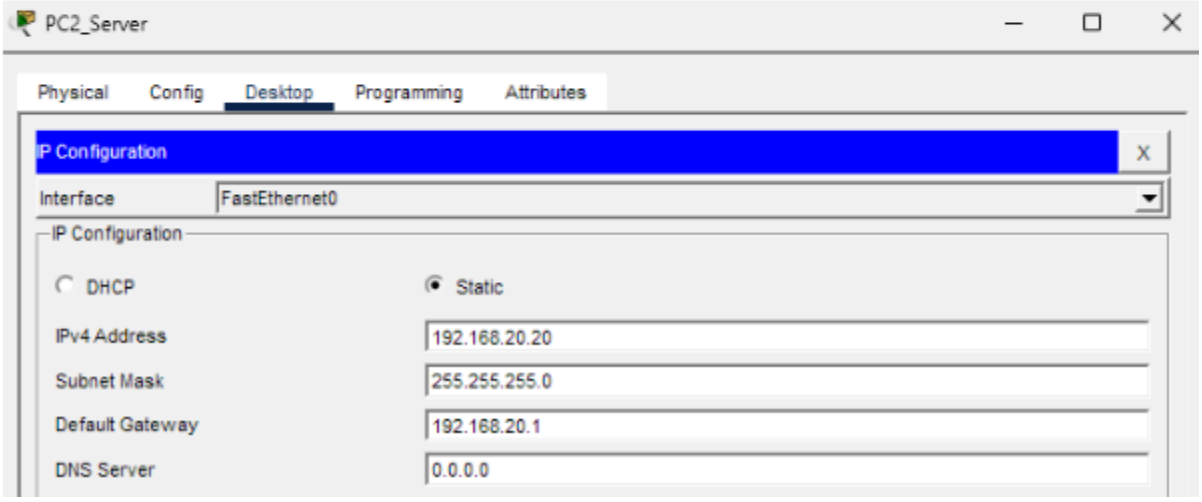
Copy Paste

```
LAN Name                             Status    Ports
--- ---------------------------------- --------- -------------------------------
    default                          active    Fa0/3, Fa0/4, Fa0/5, Fa0/6
                                               Fa0/7, Fa0/8, Fa0/9, Fa0/10
                                               Fa0/11, Fa0/12, Fa0/13, Fa0/14
                                               Fa0/15, Fa0/16, Fa0/17, Fa0/18
                                               Fa0/19, Fa0/20, Fa0/21, Fa0/22
                                               Fa0/23, Fa0/24, Gig0/2
0   CLIENTS                          active    Fa0/1
0   SERVERS                          active    Fa0/2
002 fddi-default                     active
003 token-ring-default               active
004 fddinet-default                  active
005 trnet-default                    active
```

Step 3 — Configure Hosts

Set static IP addresses per table.

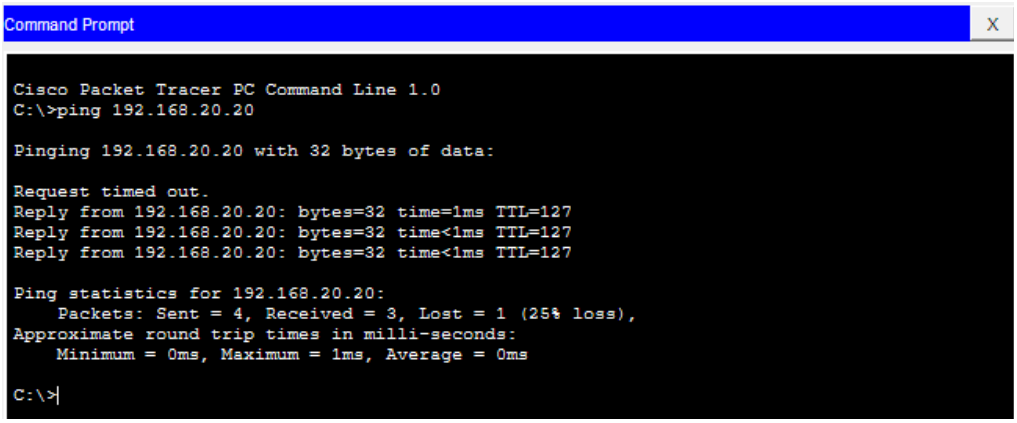




🔍 Part 2 — Verify Connectivity

From PC1:

```
None
ping 192.168.20.20
```



Verify:

- Successful replies
- ARP entries

Router checks:

```
None
show ip route
show ip interface brief
```

IOS Command Line Interface

```
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0.10, changed state to
up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0.20, changed state to
up


R1>show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0.10
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0.10
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0.20
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0.20

R1>show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  administratively down down
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R1>
```

Switch checks:

None

```
show interfaces trunk
show mac address-table
```

IOS Command Line Interface

```
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to up

%LINK-5-CHANGED: Interface FastEthernet0/2, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/2, changed state to up

%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up


S1>show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      1

Port        Vlans allowed on trunk
Gig0/1      1-1005

Port        Vlans allowed and active in management domain
Gig0/1      1,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Gig0/1      1,10,20

S1>show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    00d0.bc14.0e01    DYNAMIC     Gig0/1
  10    0090.2bee.9a87    DYNAMIC     Fa0/1
  10    00d0.bc14.0e01    DYNAMIC     Gig0/1
  20    0030.a34a.1210    DYNAMIC     Fa0/2
  20    00d0.bc14.0e01    DYNAMIC     Gig0/1
S1>
```

🧪 Part 3 — Deploy MIME Server

On Server (PC2)

```
None
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

```
PS C:\Users\piriy> cd D:\Work\network\lab3\week01-mime-typing
PS D:\Work\network\lab3\week01-mime-typing> python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
[18:27:11] INFO: Starting server with host=0.0.0.0, port=65432, storage=storage
[18:27:11] INFO: Server listening on 0.0.0.0:65432
```

On Client (PC1)

```
None
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port
65432
```

```
D:\Work\network\lab3\week01-mime-typing>python client/main_enhanced.py assets/sample.png --host 127.0.0.1 --port 65432
[18:53:14] INFO: Starting client with 1 files to 127.0.0.1:65432
[18:53:14] INFO: Connected to 127.0.0.1:65432
[18:53:14] INFO: Sent assets/sample.png (108 bytes) as image/png
[18:53:14] INFO: Connection closed. Sent: 1, Failed: 0
[18:53:14] INFO: All files sent successfully
```

Confirm file saved.

📡 Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

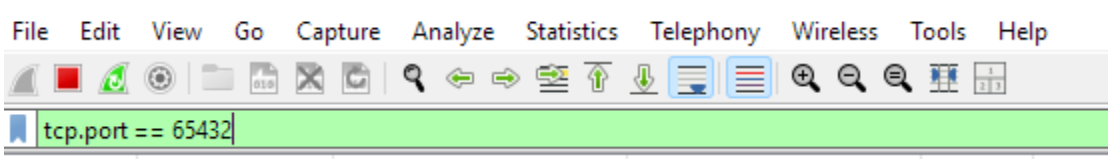
Step 1 — Start Capture

Open Wireshark.

Select interface connected to network.

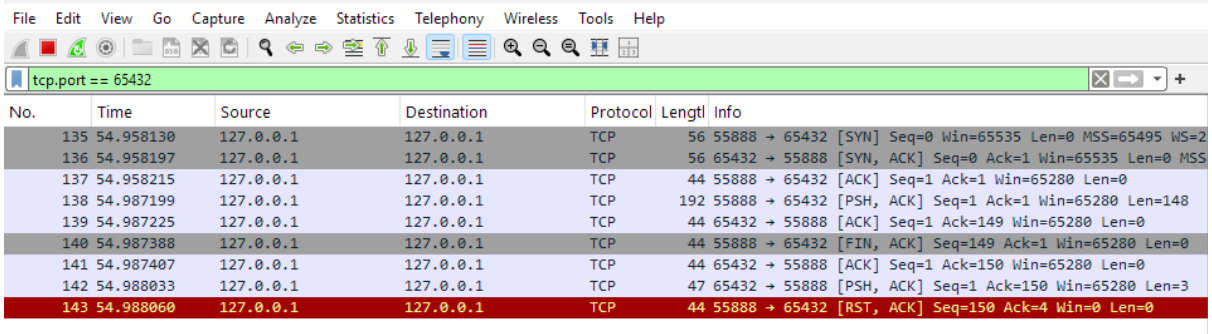
Apply capture filter:

```
None
tcp port 65432
```



Step 2 — Initiate Transfer

Send file again.



No.	Time	Source	Destination	Protocol	Lengtl	Info
135	54.958130	127.0.0.1	127.0.0.1	TCP	56	55888 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=2
136	54.958197	127.0.0.1	127.0.0.1	TCP	56	65432 → 55888 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS
137	54.958215	127.0.0.1	127.0.0.1	TCP	44	55888 → 65432 [ACK] Seq=1 Ack=1 Win=65280 Len=0
138	54.987199	127.0.0.1	127.0.0.1	TCP	192	55888 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=148
139	54.987225	127.0.0.1	127.0.0.1	TCP	44	65432 → 55888 [ACK] Seq=1 Ack=149 Win=65280 Len=0
140	54.987388	127.0.0.1	127.0.0.1	TCP	44	55888 → 65432 [FIN, ACK] Seq=149 Ack=1 Win=65280 Len=0
141	54.987407	127.0.0.1	127.0.0.1	TCP	44	65432 → 55888 [ACK] Seq=1 Ack=150 Win=65280 Len=0
142	54.988033	127.0.0.1	127.0.0.1	TCP	47	65432 → 55888 [PSH, ACK] Seq=1 Ack=150 Win=65280 Len=3
143	54.988060	127.0.0.1	127.0.0.1	TCP	44	55888 → 65432 [RST, ACK] Seq=150 Ack=4 Win=0 Len=0

Stop capture after completion.

Step 3 — Analyze Packets

A — ARP Exchange

Look for:

- Who has 192.168.20.20?
- เนื่องจากในการทดลองนี้ใช้ loopback address (127.0.0.1) (คุยกับเครื่องตัวเอง) แทนการส่งข้าม VLAN จริง และเนื่องจาก ARP ทำงานที่ Layer 2 เพื่อหา MAC address ของ host ดังนั้นจึงไม่มี ARP เกิดขึ้น

Confirm MAC resolution.

B — TCP Three-Way Handshake

Identify:

1 SYN 2 SYN-ACK 3 ACK

Protocol	Length	Info
TCP	56	55888 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
TCP	56	65432 → 55888 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
TCP	44	55888 → 65432 [ACK] Seq=1 Ack=1 Win=65280 Len=0

Check sequence numbers.

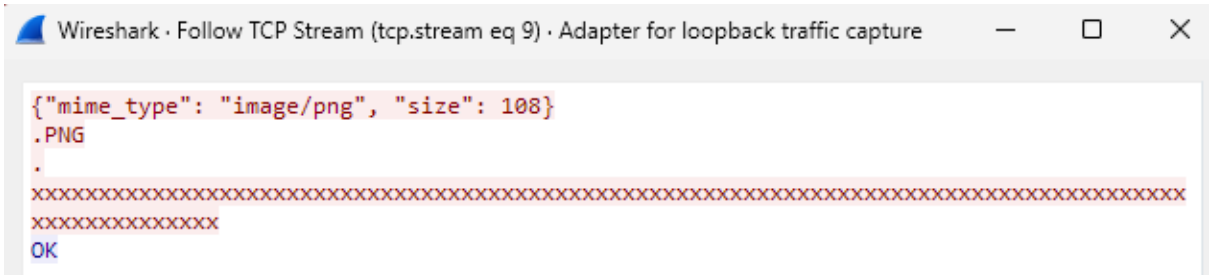
C — Data Transfer

Follow stream:

Right click → Follow → TCP Stream.

You should see JSON header similar to:

```
None
{"mime_type":"image/png","size":12345}
```


Wireshark · Follow TCP Stream (tcp.stream eq 9) · Adapter for loopback traffic capture
{"mime_type": "image/png", "size": 108}
.PNG
.
xxxx

Layer	Field	What to Observe
IP	TTL	Routing behavior
TCP	Flags	Reliability
Payload	JSON	MIME metadata

🛠️ Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

```
[18:57:26] INFO: Connection with ('127.0.0.1', 55888) closed
[19:09:12] INFO: Shutdown signal received, stopping server...
[19:09:12] INFO: Shutting down server socket
[19:09:12] INFO: Server stopped
PS D:\Work\network\lab3\week01-mime-typing>
```

Observe:

- Ping failure
- TCP timeout

```
D:\Work\network\lab3\week01-mime-typing>python client/main_enhanced.py assets/sample.png --host 127.0.0.1 --port 65432
[19:09:54] INFO: Starting client with 1 files to 127.0.0.1:65432
[19:09:56] ERROR: Failed to connect to 127.0.0.1:65432: [WinError 10061] No connection could be made because the target
machine actively refused it
[19:09:56] ERROR: Some files failed to send
```

ไม่มี server ที่เป็นผู้รับ

Scenario 2 — Firewall Block

Block port.

เนื่องจากใช้ Localhost จึงไม่โดน Firewall บล็อก ในระดับเครือข่าย

Observe:

- SYN retransmissions

Scenario 3 — Wrong Gateway

Observe:

```
Pinging 192.168.20.20 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>
```

เนื่องจากส่งข้อมูลไปยัง gateway ที่ไม่มีอยู่จรง ทำให้ข้อมูลส่งไปไม่ถึง จึงเกิด ARP incomplete

- ARP incomplete

Scenario 4 — Packet Loss Simulation

Drop packets.

No.	Time	Source	Destination	Protocol	Length	Info
25	35.192196	127.0.0.1	127.0.0.1	TCP	56	63634 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=
26	35.192265	127.0.0.1	127.0.0.1	TCP	56	65432 → 63634 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MS
27	35.192288	127.0.0.1	127.0.0.1	TCP	44	63634 → 65432 [ACK] Seq=1 Ack=1 Win=65280 Len=0
28	35.224263	127.0.0.1	127.0.0.1	TCP	192	63634 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=148
29	35.224291	127.0.0.1	127.0.0.1	TCP	44	65432 → 63634 [ACK] Seq=1 Ack=149 Win=65280 Len=0
30	35.224456	127.0.0.1	127.0.0.1	TCP	44	63634 → 65432 [FIN, ACK] Seq=149 Ack=1 Win=65280 Len=0
31	35.224474	127.0.0.1	127.0.0.1	TCP	44	65432 → 63634 [ACK] Seq=1 Ack=150 Win=65280 Len=0
32	35.227058	127.0.0.1	127.0.0.1	TCP	47	65432 → 63634 [PSH, ACK] Seq=1 Ack=150 Win=65280 Len=3
33	35.227104	127.0.0.1	127.0.0.1	TCP	44	63634 → 65432 [RST, ACK] Seq=150 Ack=4 Win=0 Len=0
46	96.094072	127.0.0.1	127.0.0.1	TCP	56	51300 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=
47	96.094134	127.0.0.1	127.0.0.1	TCP	56	65432 → 51300 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MS
48	96.094156	127.0.0.1	127.0.0.1	TCP	44	51300 → 65432 [ACK] Seq=1 Ack=1 Win=65280 Len=0
49	96.094558	127.0.0.1	127.0.0.1	TCP	44	51300 → 65432 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
50	96.094580	127.0.0.1	127.0.0.1	TCP	44	65432 → 51300 [ACK] Seq=1 Ack=2 Win=65280 Len=0
51	96.094866	127.0.0.1	127.0.0.1	TCP	44	65432 → 51300 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
52	96.094896	127.0.0.1	127.0.0.1	TCP	44	51300 → 65432 [ACK] Seq=2 Ack=2 Win=65280 Len=0

เนื่องจาก drop packet ทำให้ client พยายาม เชื่อมต่อใหม่ เพราะ connection ก่อนหน้าพัง ซึ่งแสดงให้เห็นถึงกลไกความเชื่อถือได้ (Reliability) และความสามารถในการกู้คืนการเชื่อมต่อ (Recovery Mechanism) ของโปรโตคอล TCP

นอกจากนี้ ยังสามารถสังเกตเห็นการเปลี่ยนแปลงของค่า TCP Window Size ที่ถูกประกาศระหว่างการเชื่อมต่อใหม่ ซึ่งสะท้อนถึงกระบวนการควบคุมการไหลของข้อมูล (Flow Control) และการปรับตัวของ TCP ต่อสภาวะที่เกิดการสูญหายของแพ็กเก็ต

Observe:

- TCP retransmissions
- Window adjustment

📝 Lab Report Requirements

Include:

- Screenshots of configurations
- Ping results
- Wireshark capture screenshots
- JSON header evidence
- Encapsulation explanation
- Troubleshooting notes

🎓 Assessment Questions

1. **Why does TCP retransmit lost segments?**
   เพราะ TCP เป็นโปรโตคอลที่ออกแบบมาให้มีความเชื่อถือได้ (reliable) เมื่อ TCP ตรวจพบว่า segment บางส่วนสูญหายไป (เช่น ไม่ได้รับ ACK ภายในเวลาที่กำหนด หรือได้รับ Duplicate ACK) TCP จะทำการ ส่ง segment นั้นซ้ำ (retransmission) เพื่อให้แน่ใจว่าข้อมูลถูกส่งถึงปลายทางครบถ้วนและเรียงลำดับถูกต้อง
2. **How does VLAN tagging affect broadcast domains?**
   VLAN tagging ทำให้เครือข่ายเดียวกันทางกายภาพถูกแบ่งออกเป็นหลาย broadcast domain ทางตรรกะ อุปกรณ์ใน VLAN เดียวกันเท่านั้นที่จะได้รับ broadcast แม้จะเชื่อมต่อผ่าน switch ตัวเดียวกัน แต่ต่าง VLAN จะไม่รับ broadcast ของกันและกัน ผลคือช่วยลด broadcast traffic และเพิ่มประสิทธิภาพของเครือข่าย
3. **What role does MIME play compared to TCP headers?**
   TCP header ทำหน้าที่ควบคุมการส่งข้อมูล เช่น ลำดับข้อมูล การยืนยัน (ACK) และความน่าเชื่อถือ
   MIME ทำหน้าที่อธิบาย ความหมายของข้อมูล ในระดับแอปพลิเคชัน เช่น ชนิดไฟล์ (image/png, text/html)

4. **Where does routing decision occur?**
   Layer 3 โดย Router จะใช้ IP address และ routing table เพื่อตัดสินใจว่า packet ควรถูกส่งต่อไปทางใด
5. **Why does ARP precede TCP communication?**
   ก่อนที่ TCP จะเริ่มทำงาน อุปกรณ์ต้องรู้ **MAC address ของปลายทางในเครือข่ายท้องถิ่น** ARP ทำหน้าที่แปลง IP address เป็น MAC address เมื่อได้ MAC address แล้ว จึงสามารถส่ง frame ในระดับ Data Link Layer ได้ ดังนั้น ARP จึงจำเป็นต้องเกิดก่อน TCP

📊 Reflection Exercise

Explain:

How does application meaning survive multiple encapsulation layers?

แม้ข้อมูลจะถูกห่อหุ้ม (encapsulated) ด้วย header หลายชั้น เช่น TCP, IP และ Ethernet แต่ payload ของชั้นแอปพลิเคชันจะไม่ถูกเปลี่ยนแปลง ระหว่างการส่ง
แต่ละชั้นของเครือข่ายมีหน้าที่เฉพาะของตนเอง และจะอ่านหรือแก้ไขเฉพาะ header ของชั้นนั้นเท่านั้น
เมื่อข้อมูลถึงปลายทาง กระบวนการ decapsulation จะค่อย ๆ แกะ header ออก
จนเหลือข้อมูลเดิมที่แอปพลิเคชันสามารถตีความได้อย่างถูกต้อง

⚠️ Common Pitfalls

- Server bound to localhost only
- Missing trunk
- Wrong VLAN
- OS firewall enabled
- Capture on wrong interface

🚀 Extension Activities

- Add TLS encryption
- Measure throughput vs latency
- Capture checksum errors
- Add checksum field to protocol
- Containerize server
- Compare with HTTP transfer

🧬 Instructor Notes — What This Lab Really Teaches

This is not just networking.

It demonstrates:

- Protocol layering philosophy

- Reliability engineering
- Metadata semantics
- Distributed system fundamentals

The student watches a file become:

intention → bytes → frames → signals → bytes → meaning

That transformation is the essence of computer networks.